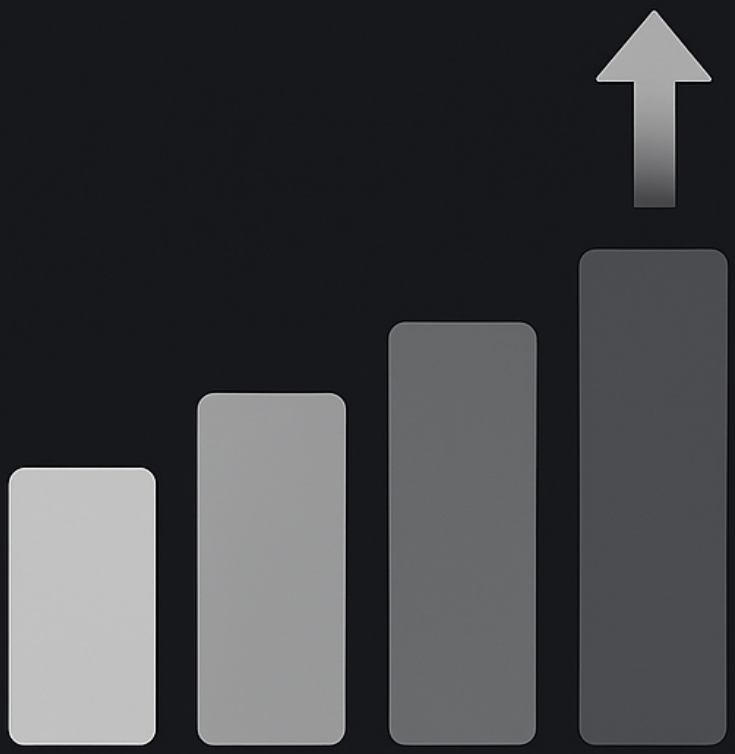
# DATA STRUCTURES AND ALGORITHMS



# SORTING

# C {...}

ai

# 336. Criteria used for analysing Sorts

1. Number of comparisions — decides time complexity
2. Number of swaps
3. Adaptive — If elements are already sorted.
4. Stable
5. Extra Memory

Ex:

| A | B | C | D |
|---|---|---|---|
| 5 | 6 | 6 | 3 |

Sorting →

| D | A | B | C |
|---|---|---|---|
| 3 | 5 | 6 | 6 |

→ order is preserved for duplicate items.

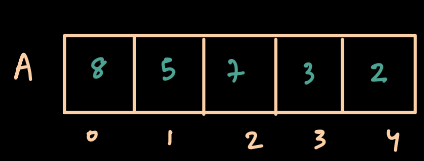
Hence stable

If

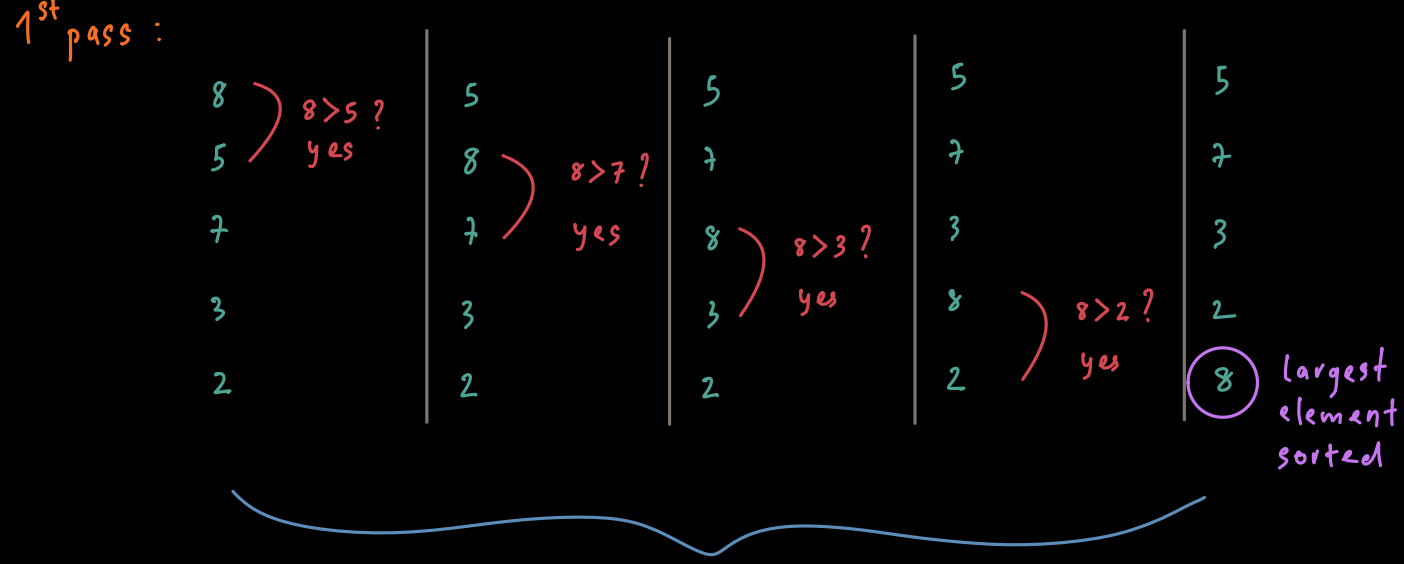
| C | B |
|---|---|
| 6 | 6 |

→ Not stable

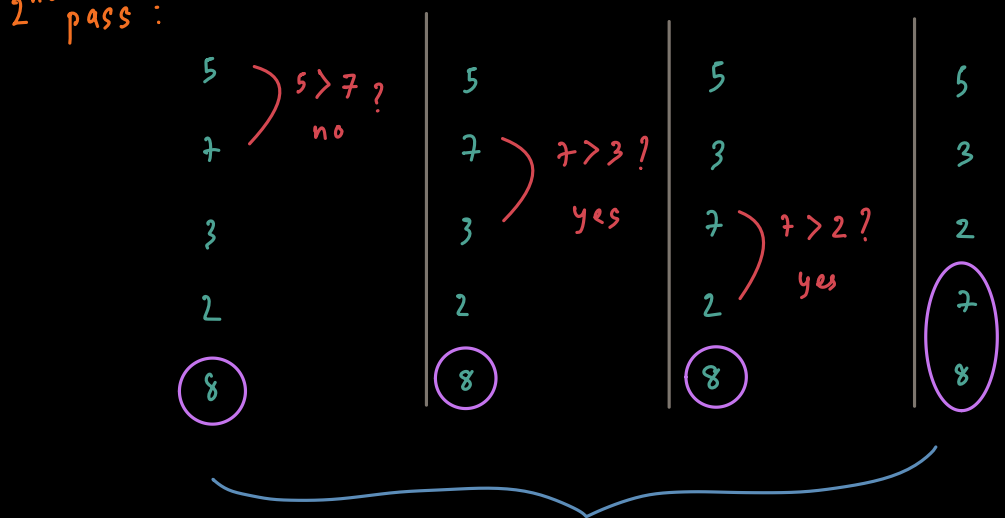
# 337. Bubble Sort




$n = 5$

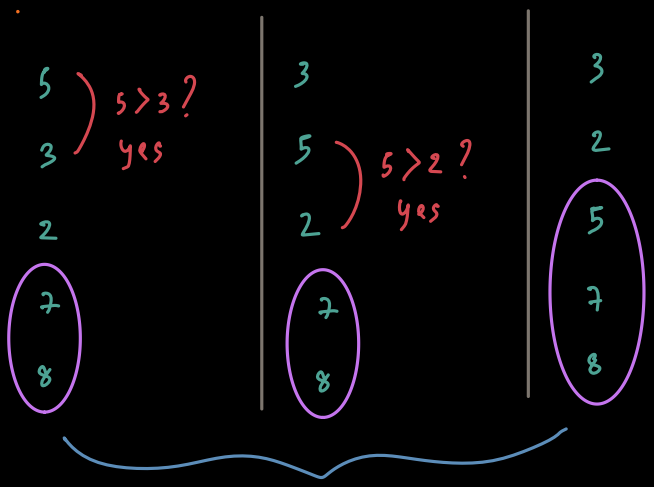



4 comparisions
4 swaps



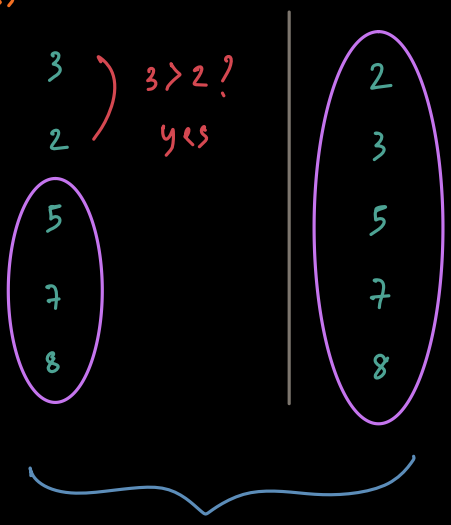

3 comparisions
3 swaps (max.)

3rd pass :




2 comparisions
2 swaps

4th pass




1 comparisions
1 swaps

* no. of passes = $(n-1)$ ; n = no. of elements

ie : passes = 4

* no. of comparisions = $1+2+\ldots+n-1$

$$= \frac{n(n-1)}{2}$$

$$= O(n^2)$$

* max. no. of swaps $= 1 + 2 + \ldots + n-1$

$= \frac{n(n-1)}{2}$

$= O(n^2)$

Time complexity of bubble sort based on no. of comparisions.

So, $O(n^2)$

## CODE :

```
void BubbleSort ( int A[], int n ) {
    int flag;
    for ( int i = 0; i < n-1; i++ ) {
        flag = 0;
        for ( int j = 0; j < n-1-i; j++ ) {
            if ( A[j] > A[j+1] ) {
                swap ( A[j], A[j+1] );
                flag = 1;
            }
        }
        if ( flag == 0 )
            break;
    }
}
```

i was subtracted coz with each iteration we shouldnot compare already sorted elements

## Bubble Sort Time

Max = $O(n^2)$

Min = $O(n)$

↳ when already sorted.


So, Bubble sort is made adaptive by using a variable flag.

* Is it stable ? ✓

Ex:

| | | |
|---|---|---|
| 8 | is 8 > 8 ? No | 8 |
| 8 | | 8 |
| 3 | | 3 |
| 5 | | 5 |
| 4 | | 4 |

Same order established

So, stable

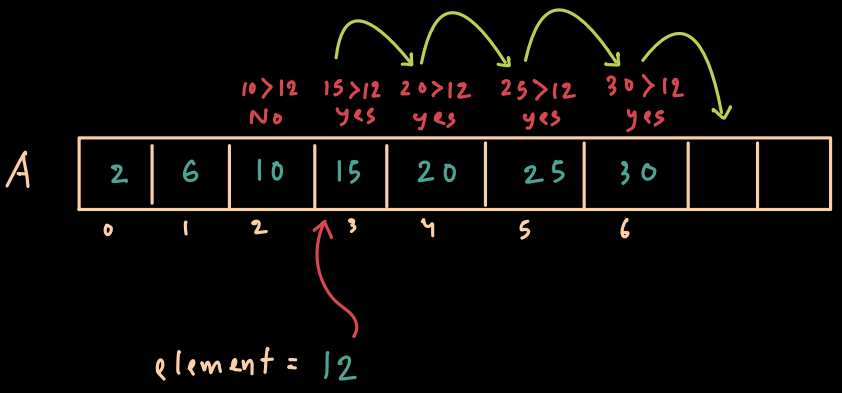
# 339. Insertion Sort

(1) In Array :

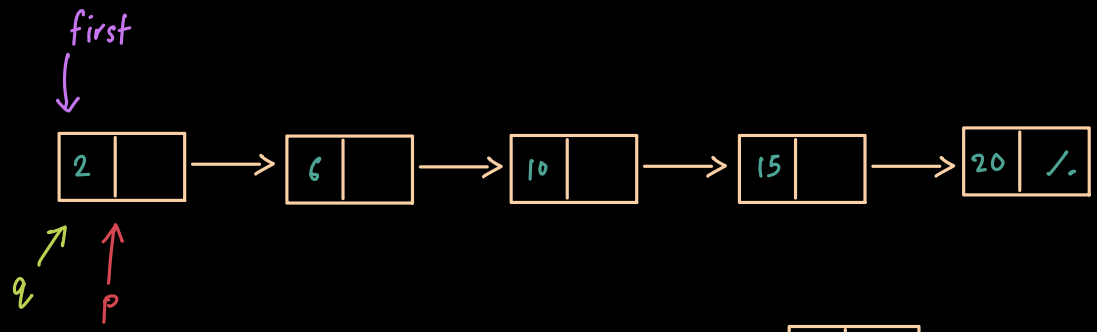
$O(n)$

↓

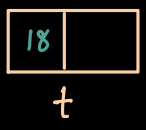
min - $O(1)$




(2) Using linked list — $O(n)$ → min $O(1)$




Element = 18




- Take two pointers p and q where q trails behind p.
- For n nodes,
- If p -> data < t -> data

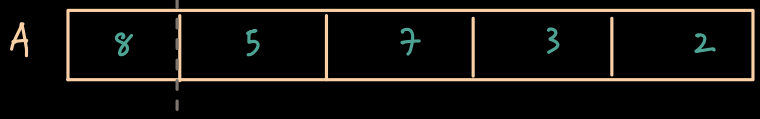
  then, bring q to p

  and, move p to next node.
- Else

  t -> next = p

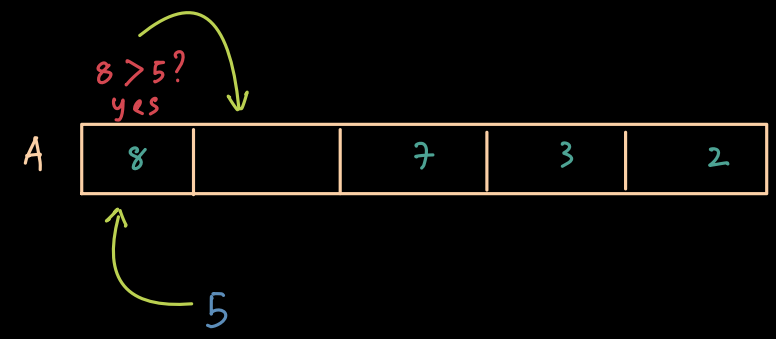
  q -> next = t

Ex :



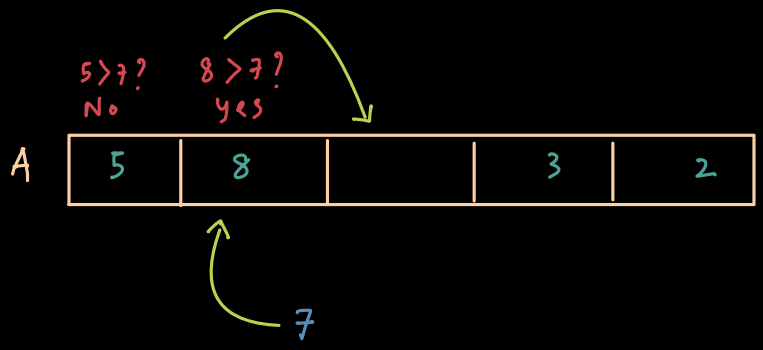

1) Insert 5

maximum



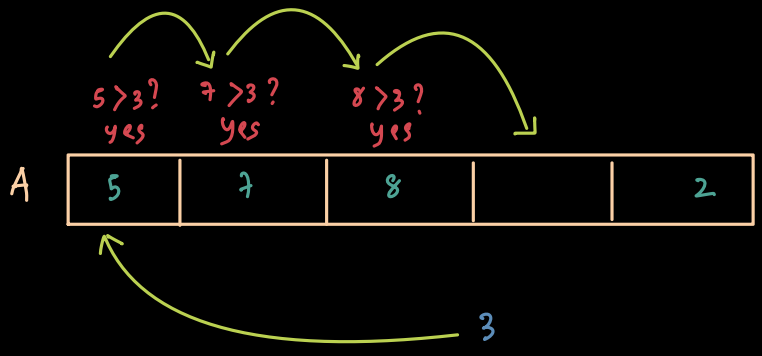

1 comp
1 swap


2) Insert 7



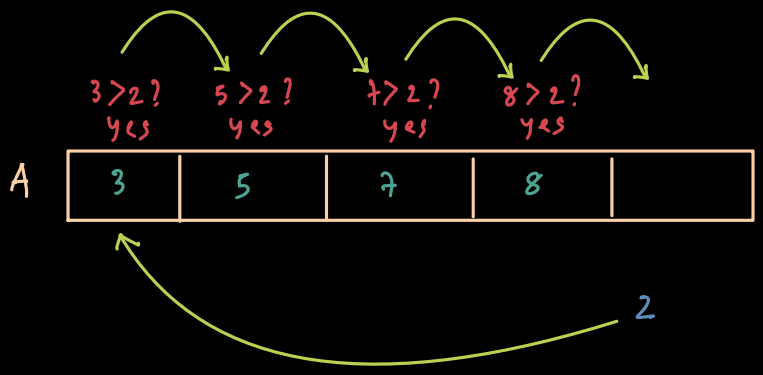

2 comp
2 swap


3) Insert 3




3 comp
3 swap


4) Insert 2




} 4 comp
4 swap


* no. of passes = $n-1$ ; n = no. of elements

5 element ⟶ 4 passes

n element ⟶ n-1 passes

* no. of comparisions = $\frac{n(n-1)}{2}$ ⟶ $O(n^2)$ (max)

5 elements ⟶ 1+2+3+4 comp.

n elements ⟶ 1+2+3+4+...+n-1 comp.

= $\frac{n(n-1)}{2}$ comp.

* no. of swaps = $\frac{n(n-1)}{2}$ ⟶ $O(n^2)$ (max)

5 elements ⟶ 1+2+3+4 swaps

n elements ⟶ 1+2+3+4+...+n-1 swaps

= $\frac{n(n-1)}{2}$ swaps

* Insertion sort is more efficient when linked list is used, as shifting doesn't happen.

## 341. Program for Insertion Sort

```
void InsertionSort(int A[], int n){
    for(int i=1; i<n; i++){
        int j = i-1;
        int x = A[i];
        while(j>-1 && A[j] > x){
            A[j+1] = A[j];
            j--;
        }
        A[j+1] = x;
    }
}
```

Assuming element at index 0 is already sorted.

j decrements and becomes −1. So while loop stops

## 342. Analysis of Insertion Sort

* For every element, there is 1 comparision

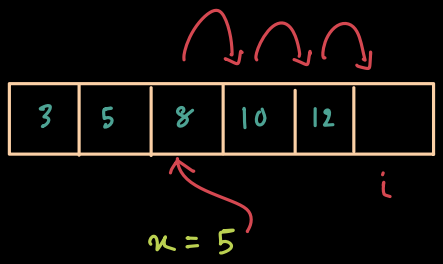
So, no. of comparisions = $n-1 \rightarrow O(n)$

* No. of swaps = 0 $\rightarrow O(1)$ (min. time)
$\rightarrow O(n^2)$ (If list is reversed)

So, Insertion Sort is ADAPTIVE (by nature, no flags req.)

STABLE ?

Ex:



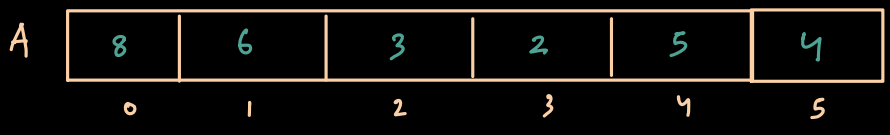


So, Insertion Sort is STABLE

## 344. Comparing Bubble Sort & Insertion Sort

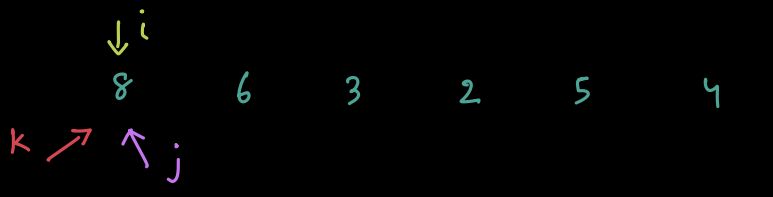
| | Bubble Sort | Insert$^n$ Sort | Case |
|---|---|---|---|
| min. Comparisions | $O(n)$ | $O(n)$ | ascending |
| max. Comparisions | $O(n^2)$ | $O(n^2)$ | descending |
| min. Swap | $O(1)$ | $O(1)$ | ascending |
| max. Swap | $O(n^2)$ | $O(n^2)$ | descending |
| Adaptive | ✓ | ✓ | - |
| Stable | ✓ | ✓ | - |
| linked list | × | ✓ | - |
| K passes | ✓ | × | - |

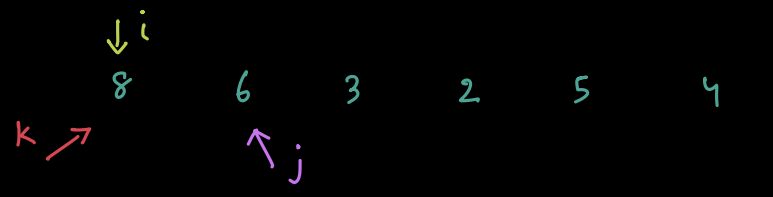
# 345. Selection Sort



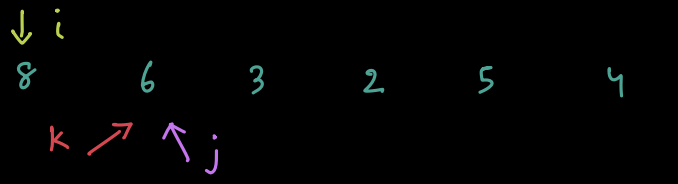

Pass 1 )

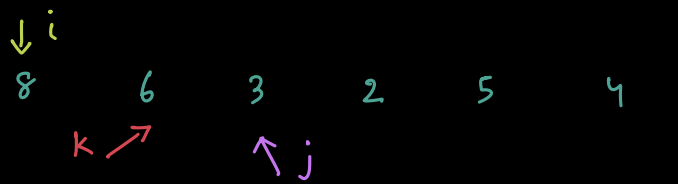
5 Comp.
1 swap



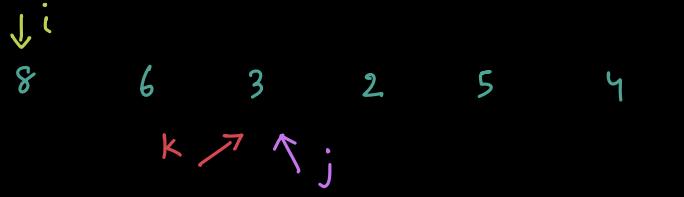



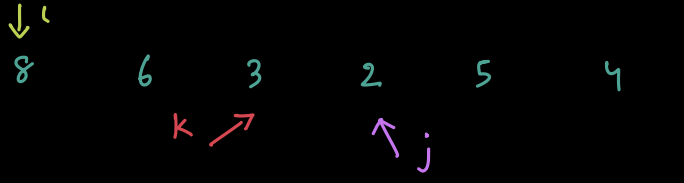

is 6 < 8 ?
yes



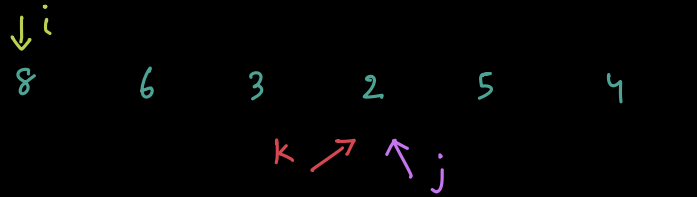



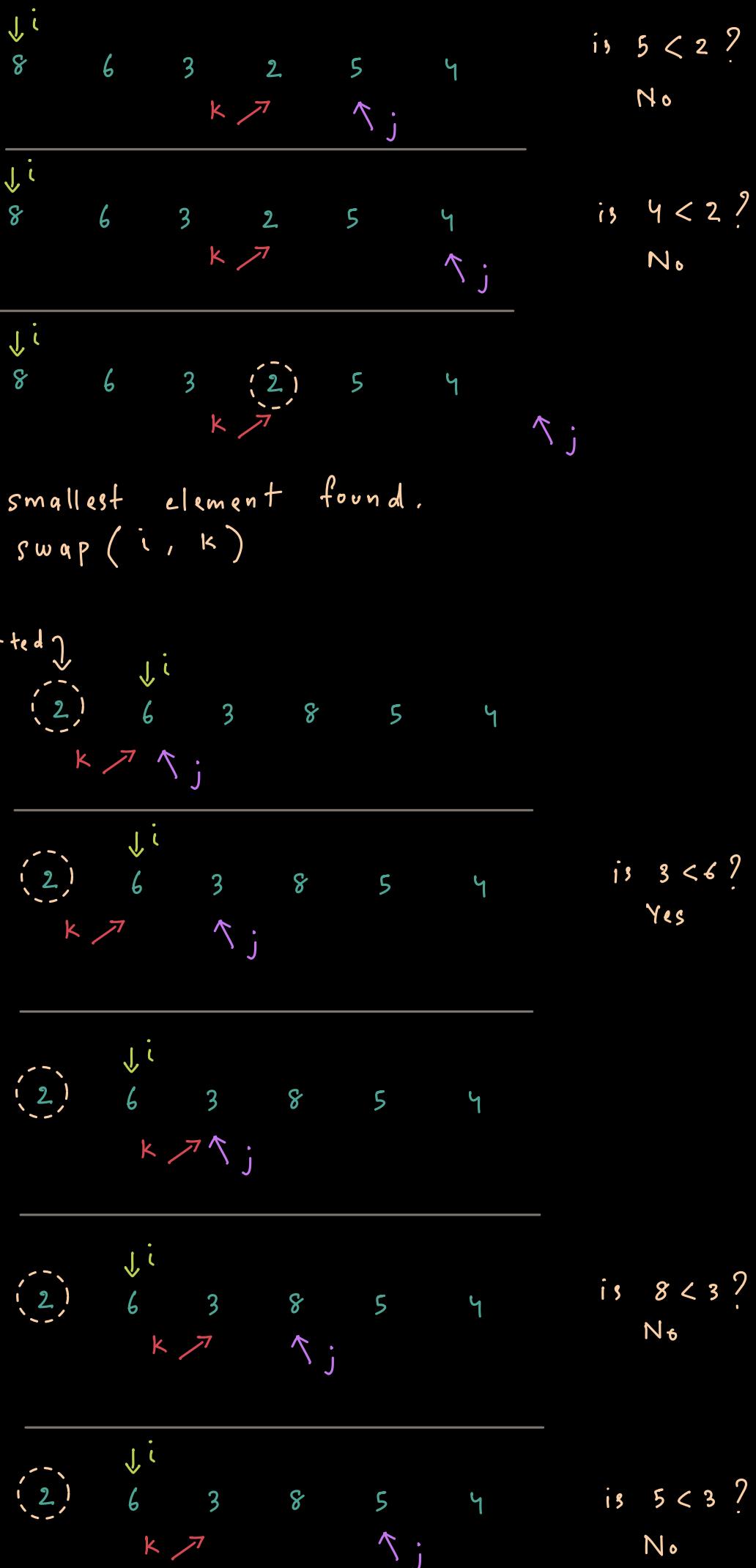

is 3 < 6 ?
yes

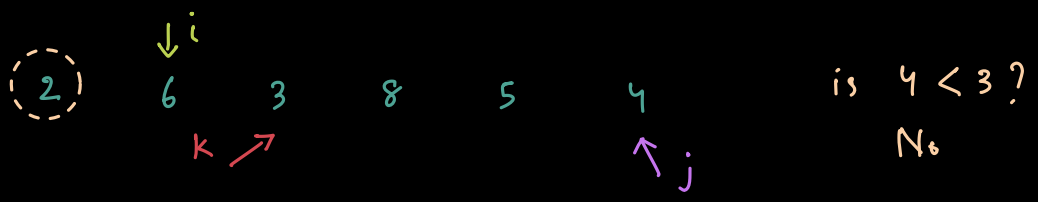



is 2 < 3 ?
yes




i
8 6 3 2 5 4
k
j
is 5 < 2 ?
No
i
8 6 3 2 5 4
k
j
is 4 < 2 ?
No
i
8 6 3 2 5 4
k
j
smallest element found.
swap ( i , k )
Pass 2)
sorted
4 Comp.
1 swap
2 6 3 8 5 4
is 3 < 6 ?
Yes
is 8 < 3 ?
No
is 5 < 3 ?
No

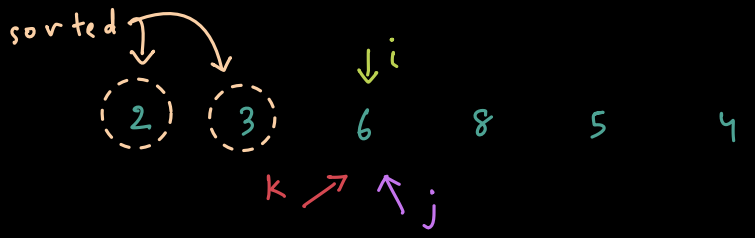



---


Smallest element found

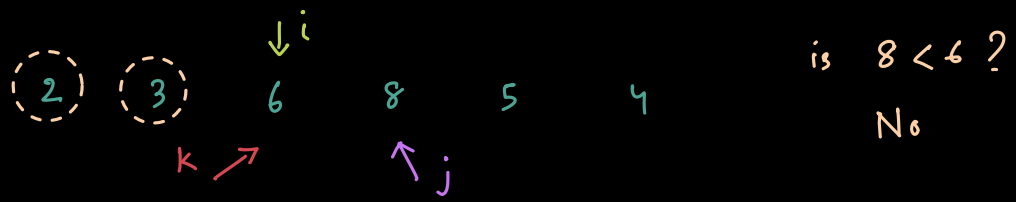
swap ( i, k )


Pass 3 )

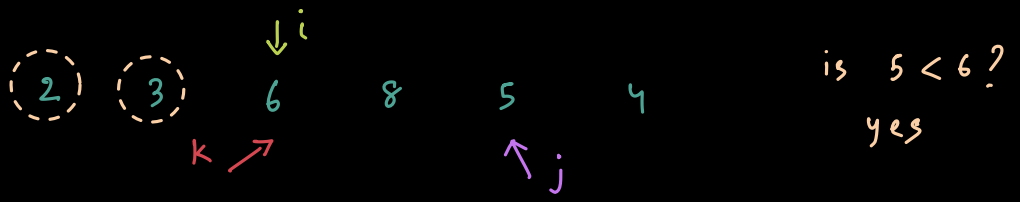



3 Comp.
1 swap

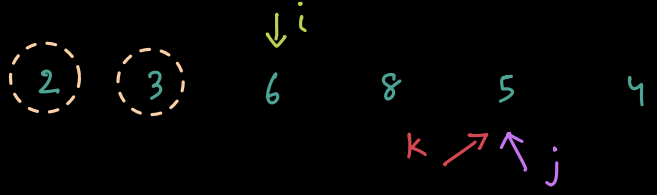
---

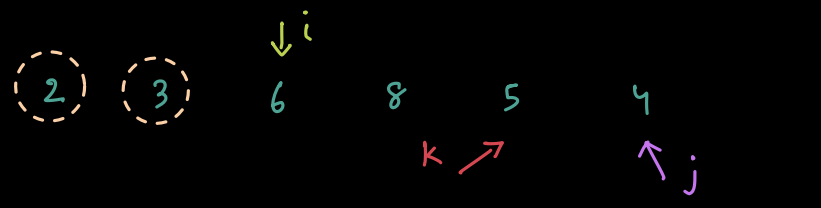



---

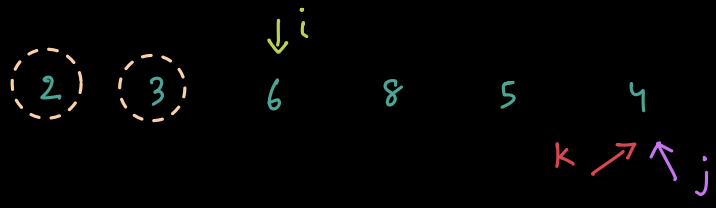



---

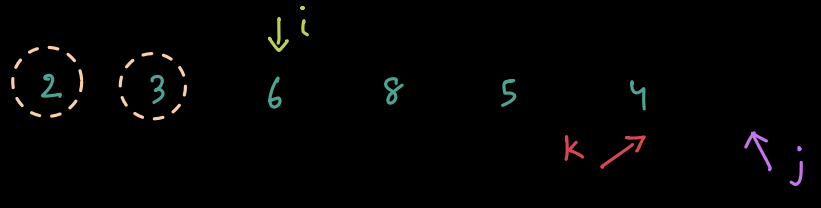





---




---




---

Smallest element found

swap(i, k)


Do this iteratively.

upto 1 comp.
1 swap.

* No. of comparisions = $\frac{n(n-1)}{2}$ $\longrightarrow O(n^2)$

↳ ie : $1+2+3+\ldots+n-1$

* No. of swaps = $n-1$ $\longrightarrow O(n)$

Only Algo that sorts in min. no. of swaps. $(n-1)$

* like Bubble sort where we get 'k' no. of smaller elements for 'k' no. of passes,

  Here we get 'k' no. of Larger elements for 'k' no. of passes.

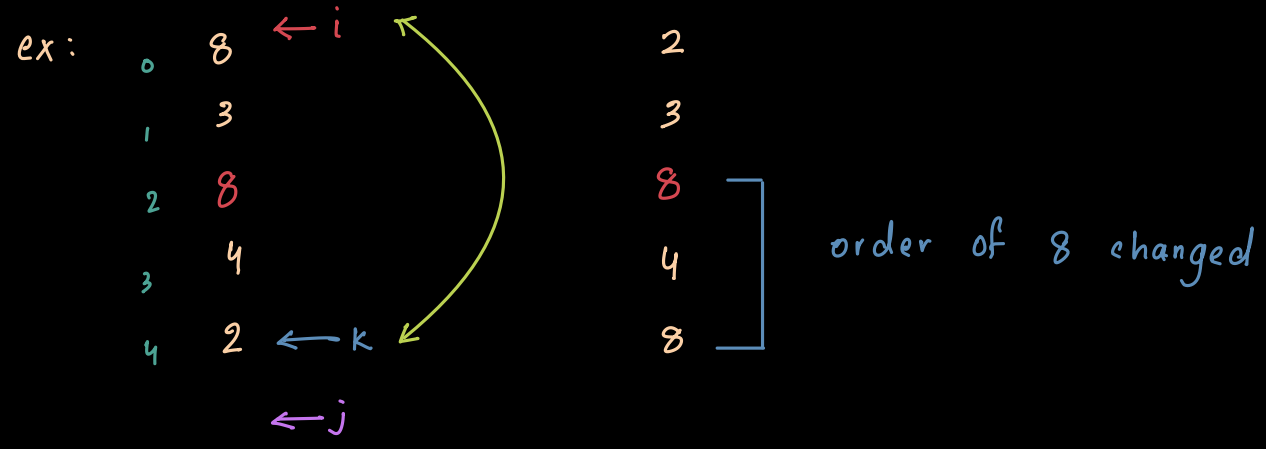
## 346. Program for Selection Sort

```
void SelectionSort ( int A[], int n ){
    int i, j, k ;
    for ( i=0 ; i<n-1 ; i++ ){
        for ( j=k=i ; j<n ; j++ ){
            if( A[j] < A[k] )
                k = j ;
        }
        swap (A[i], A[k] );
    }
}
```

## 347. Analysis of Selection Sort

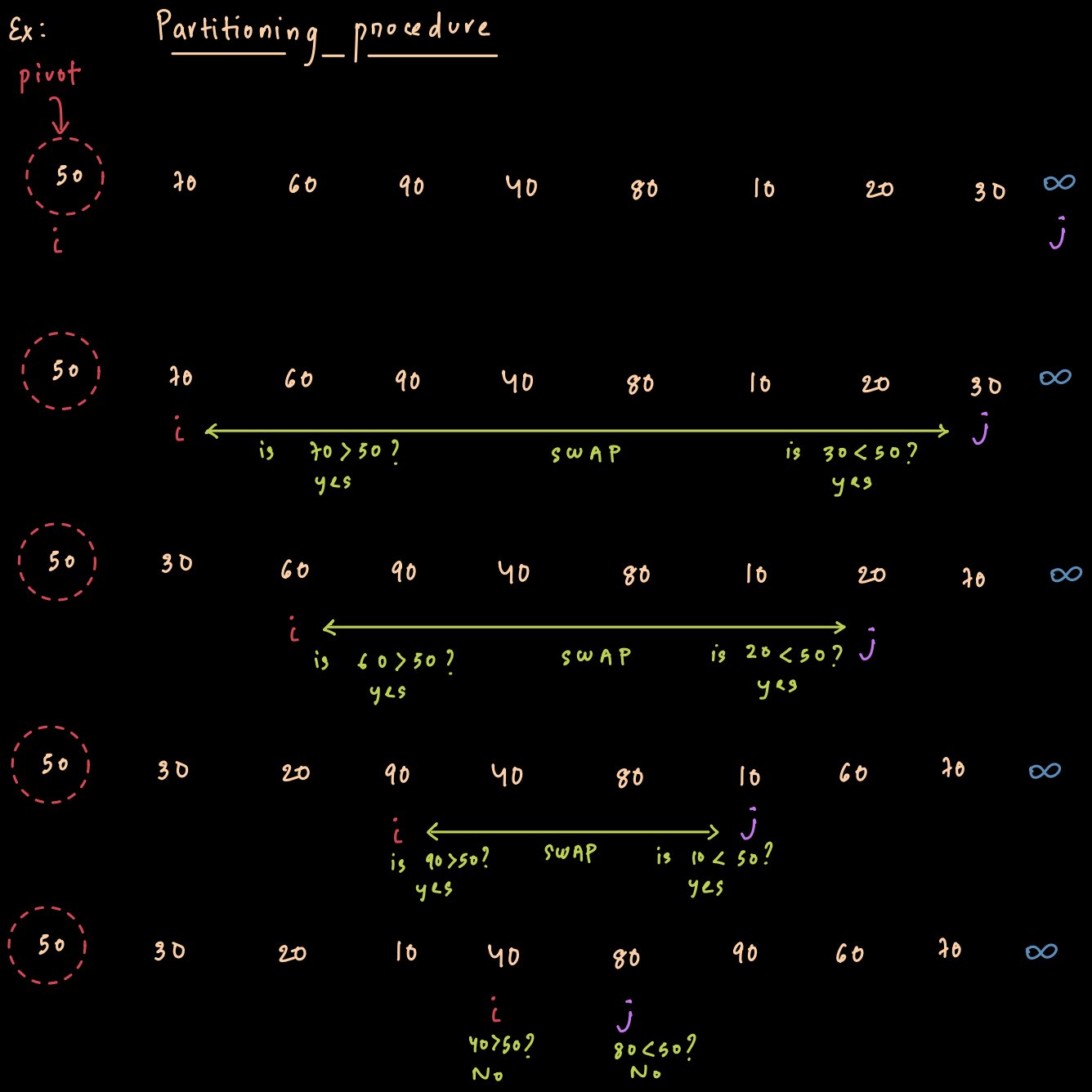
* Selection sort is NOT ADAPTIVE because no matter what it will always take time $O(n^2)$

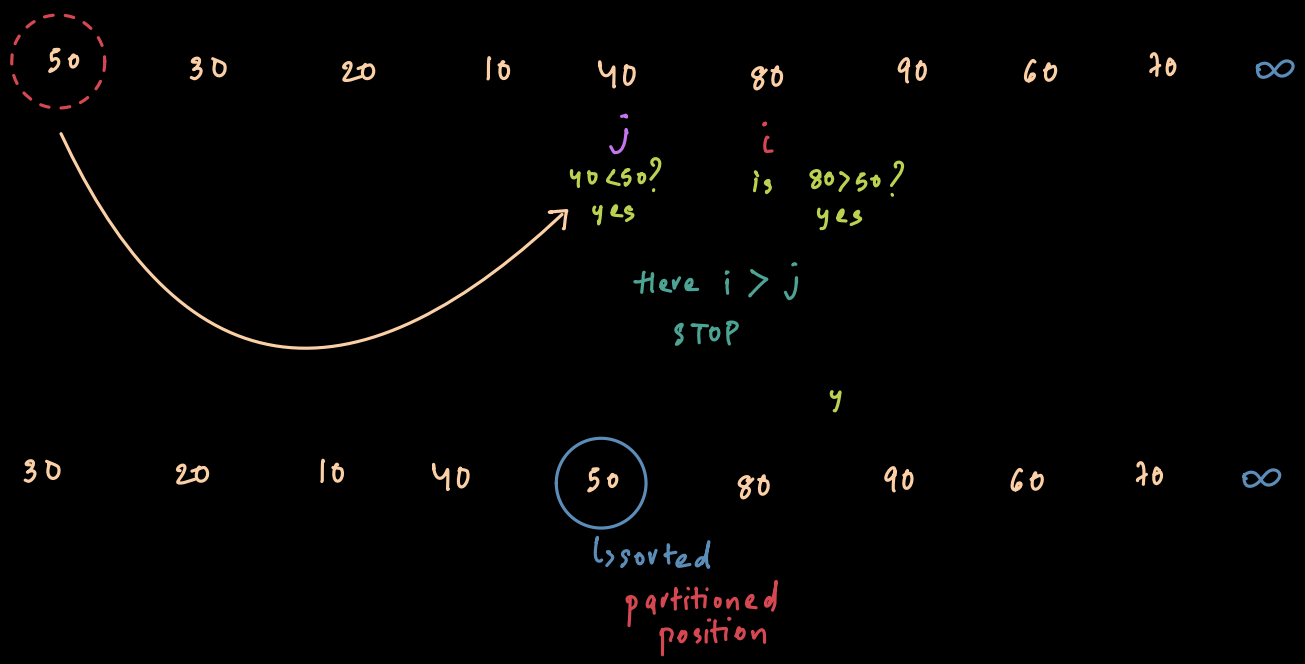
* Is it stable?




NOT STABLE

## 349. Quick Sort

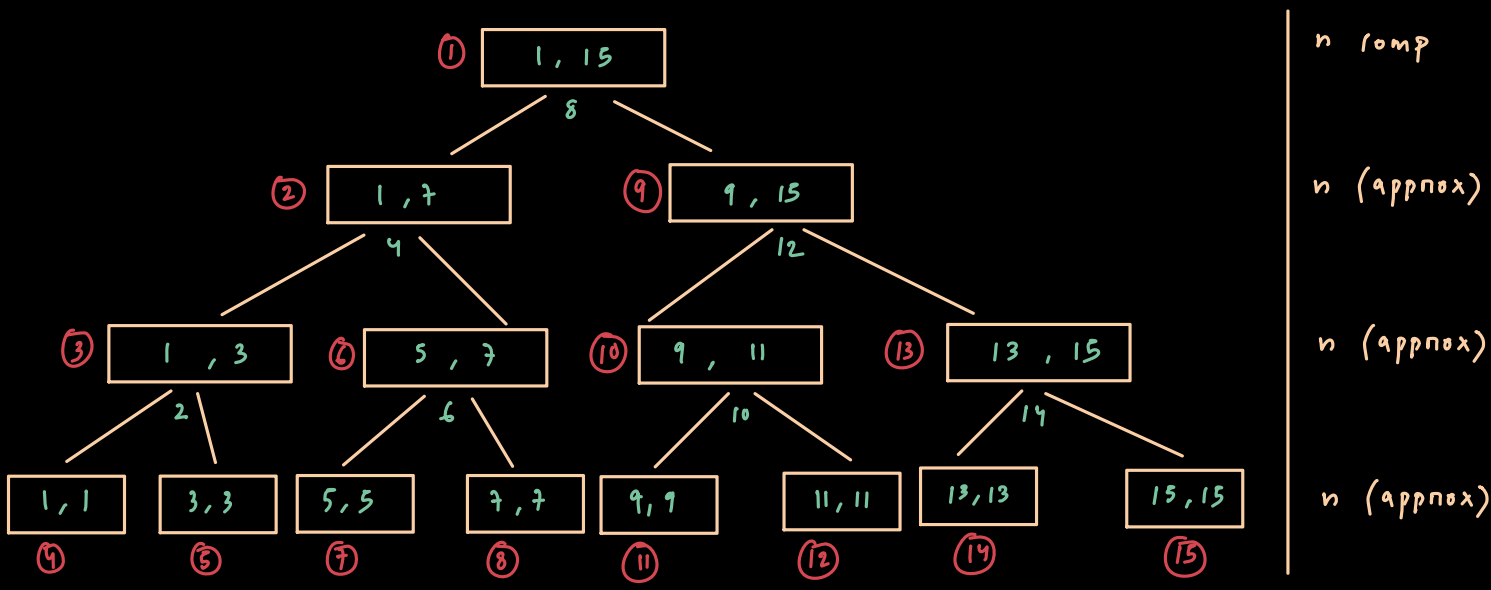





* Time complexity of Quick Sort = $\frac{n(n+1)}{2}$ = $O(n^2)$ (worst case)

* Best case time complexity :-

Every partitioning happens in the middle.




Assume total 16 elements ⟶ $\frac{16}{2} = \frac{8}{2} = \frac{4}{2} = \frac{2}{2} = 1$

So, time complexity = $O(\log_2 16)$

For, n elements = $O(\log_2 n)$

But we do appnox. n comp. at each step

So, $O(n \log_2 n)$

Time Complexity :-

1) Best case : partitioning in middle
$O(n \log_2 n)$

2) Worst case : partitioning on any one end
$O(n^2)$

} Average → $O(n \log_2 n)$

## CODE :- (1) Hoare's Partition scheme : (fewer swaps faster)

```
int Partition (int A[], int l, int h) {
    int pivot = A[l];
    int i = l, j = h;

    do {
        do { i++; } while (A[i] <= pivot);
        do { j--; } while (A[j] > pivot);

        if (i < j)
            swap (A[i], A[j]);
    }
    while (i < j);
    swap (A[l], A[j]);
    return j;
}
```

```
int    QuickSort(int A[], int l, int h){
    int j;
    if(l<h){
        j= Partition(A,l,h);
        QuickSort(A,l,j);
        QuickSort(A,j+1,h);
    }
}
```

## (2) Lomuto's Partition scheme : (Simpler & more intuitive)

```
int partition(int A[], int l, int h){
    int pivot = A[h];
    int i = l-1;

    for(int j=l; j<h; j++){
        if(A[j]<pivot){
            i++;
            Swap(A[i],A[j]);
        }
    }
    Swap(A[i+1],A[h]);
    return i+1;
}
```

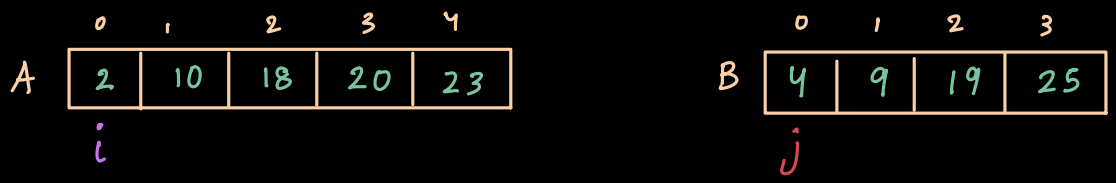
```
int    QuickSort(int A[], int l, int h){
    int j;
```

```
if ( l < h ) {
    j = Partition ( A, l, h );
    QuickSort ( A, l , j ) ;
    QuickSort ( A, j+1, h );
}
}
```

## 354. Merging

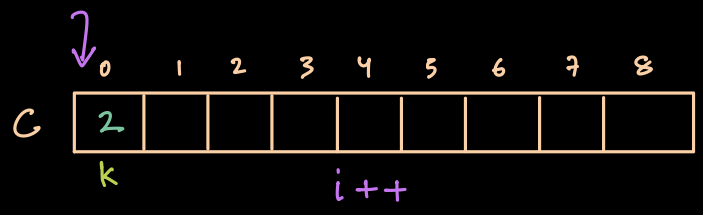
Ex)

(1)




is A[i] < A[j] ?

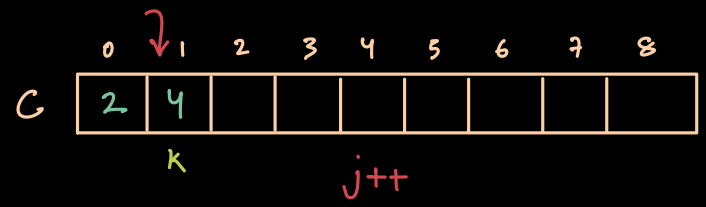
Yes



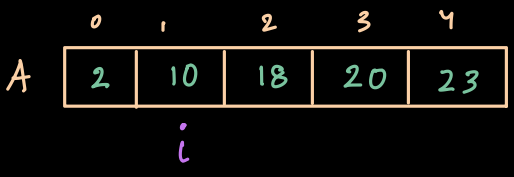

(2)

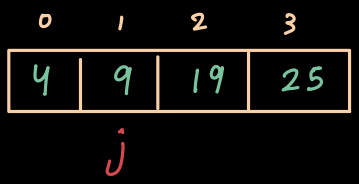

is A[i] < A[j] ?

NO



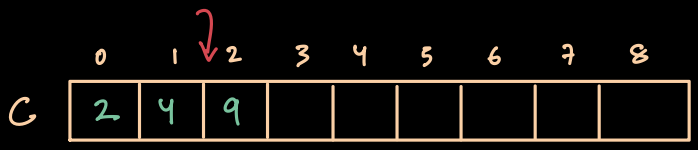
(3)



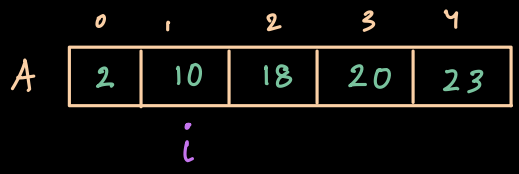



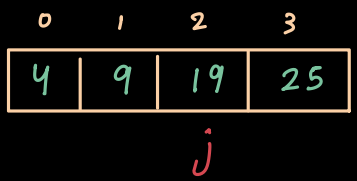

is $A[i] < A[j]$ ?

NO



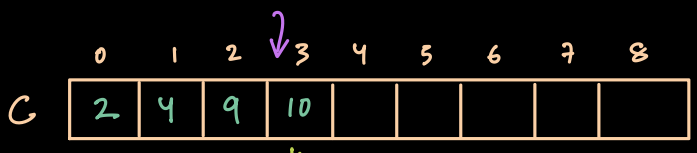

(4)



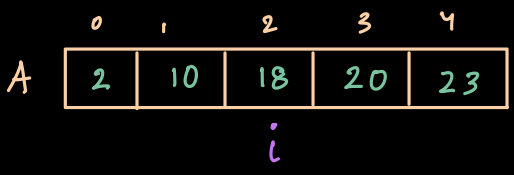



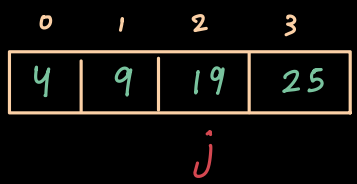

is $A[i] < A[j]$ ?

YES



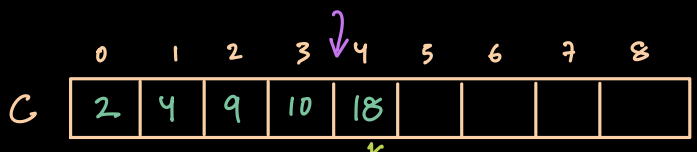

(5)



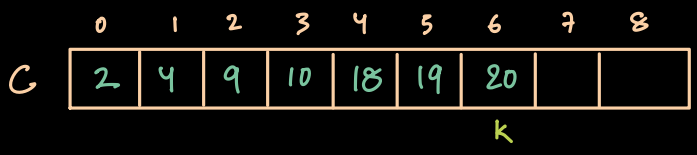



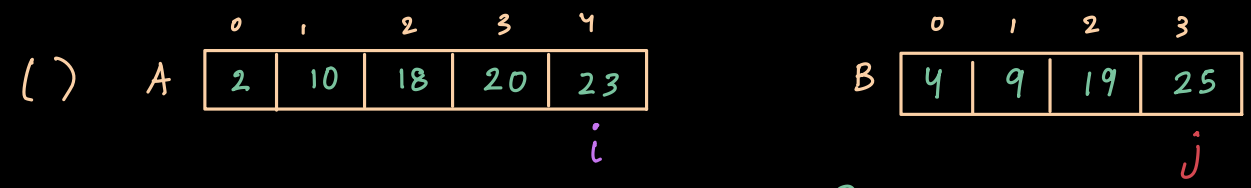

is $A[i] < A[j]$ ?

YES



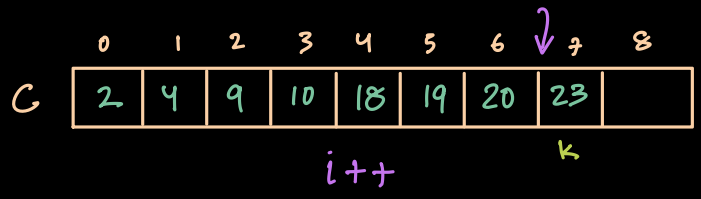

- 
- 
- 
- 

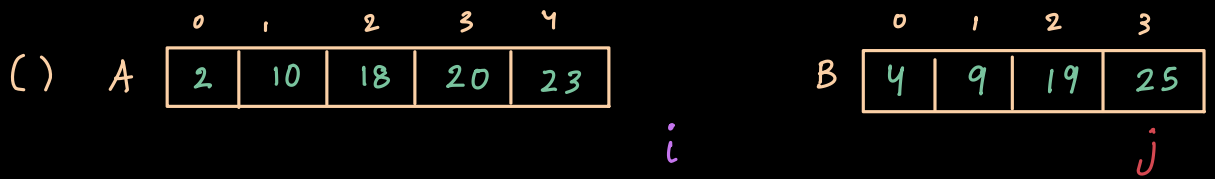





is A[i] < A[j] ?

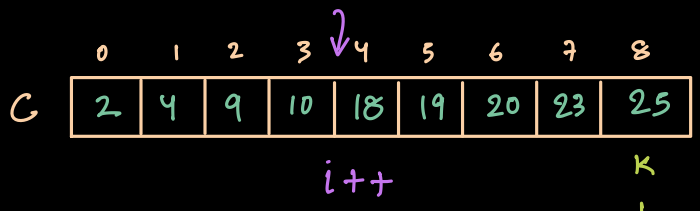
YES




is A[i] < A[j] ?

YES




One of the array will be completed first. ie : A[]

So, All the remmaining elements of other array ie : B[]

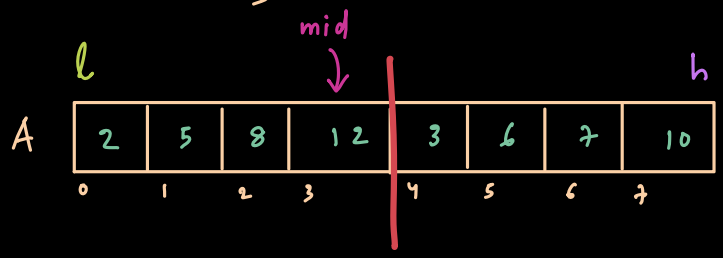
will be copied to merge array ie : C[]

CODE :

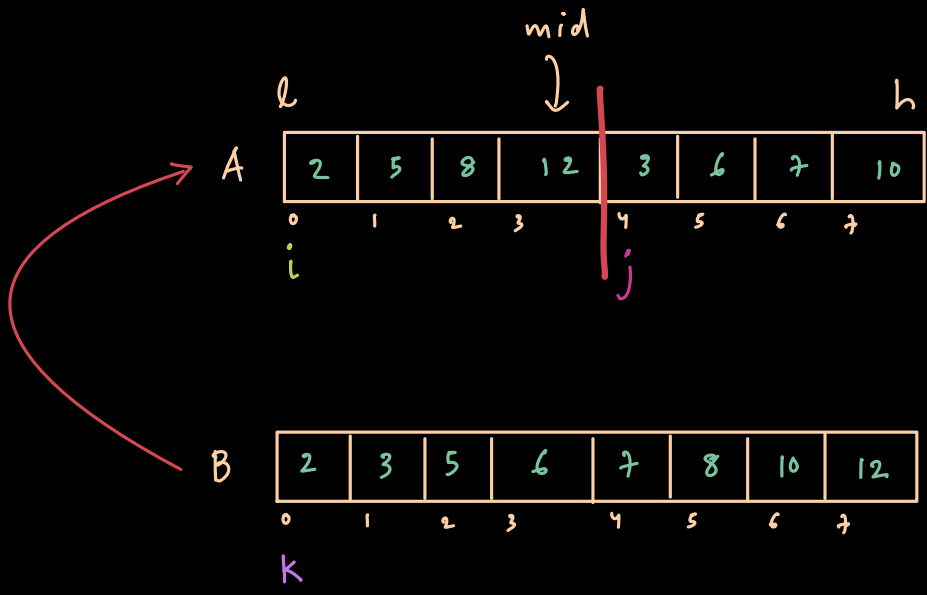
```
void Merge ( int A[], int B[], int m, int n ){
        int i, j, k ;
        i = j = k = 0 ;
        while ( i < m && j < n ){
                if ( A[i] < B[j] )
                        c[k++] = A[i++] ;
                else
                        c [k++] = B[j++] ;
        }
        // For remmaining elements
        For ( ; i<m ; i++ )          // i initialised to wherever
                c[k++] = A[i ];          it is right now.

        For ( ; j<m ; j++ )          // j initialised to wherever
                c[k++] = B[j ];          it is right now.
}
```

* Time complexity --> O(m+n)

* when instead of two arrays, only 1 array (A[]) contains 2 sorted arrays and we perform merge sort on A only.




* Here we create an auxillary array B and perform merge sort.
* Then we should copy all the sorted elements from B back to A.




CODE :

```
void Merge (int A[], int l, int mid, int h){
        int i, j, k ;
        int B[h+1] ;
        i = k = l ;
        j = mid +1 ;
```

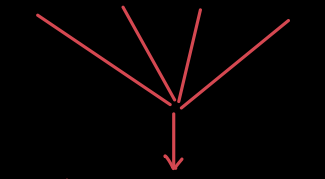
```
    while ( i <= mid  &&  j <= h ) {
        if ( A[i] < A[j] )
            B[k++] = A[i++];
        else
            B[k++] = A[j++];
    }
    // For remmaining elements
    For ( ; i <= mid ; i++ )        // i initialised to wherever
        B[k++] = A[i];                 it is right now.

    For ( ; j <= h ; j++ )          // j initialised to wherever
        B[k++] = A[j];                 it is right now.

    for ( i = L ; i <= h ; i++ )
        A[i] = B[i];
}
```

* Time complexity → $O(m+n)$

# MERGING MULTIPLE SORTED LIST

## (1) m-way merging (4-way merging)

| A | B | C | D |
|---|---|---|---|
| 2 | 3 | 5 | 8 |
| 5 | 6 | 9 | 16 |
| 15 | 18 | 12 | 20 |

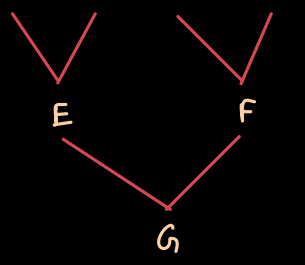


take all 4 lists together and sort them directly into a new array.

## (2) 2-way merging

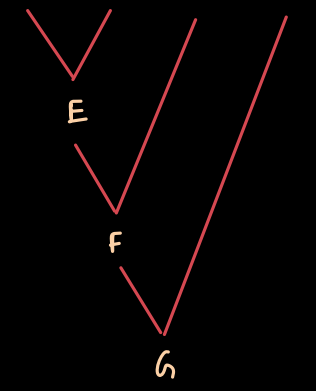
— Can be implemented iteratively or recursively

| A | B | C | D |
|---|---|---|---|
| 2 | 3 | 5 | 8 |
| 5 | 6 | 9 | 16 |
| 15 | 18 | 12 | 20 |

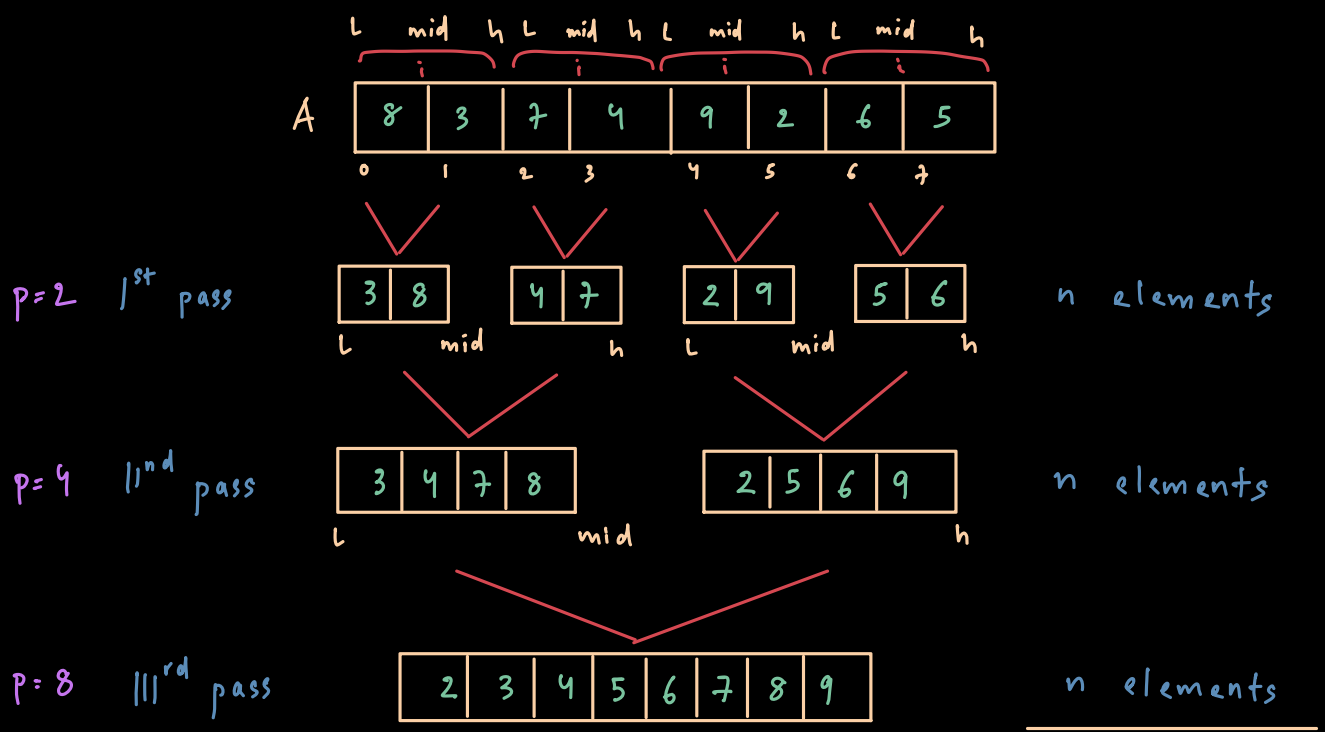



or

| A | B | C | D |
|---|---|---|---|
| 2 | 3 | 5 | 8 |
| 5 | 6 | 9 | 16 |
| 15 | 18 | 12 | 20 |



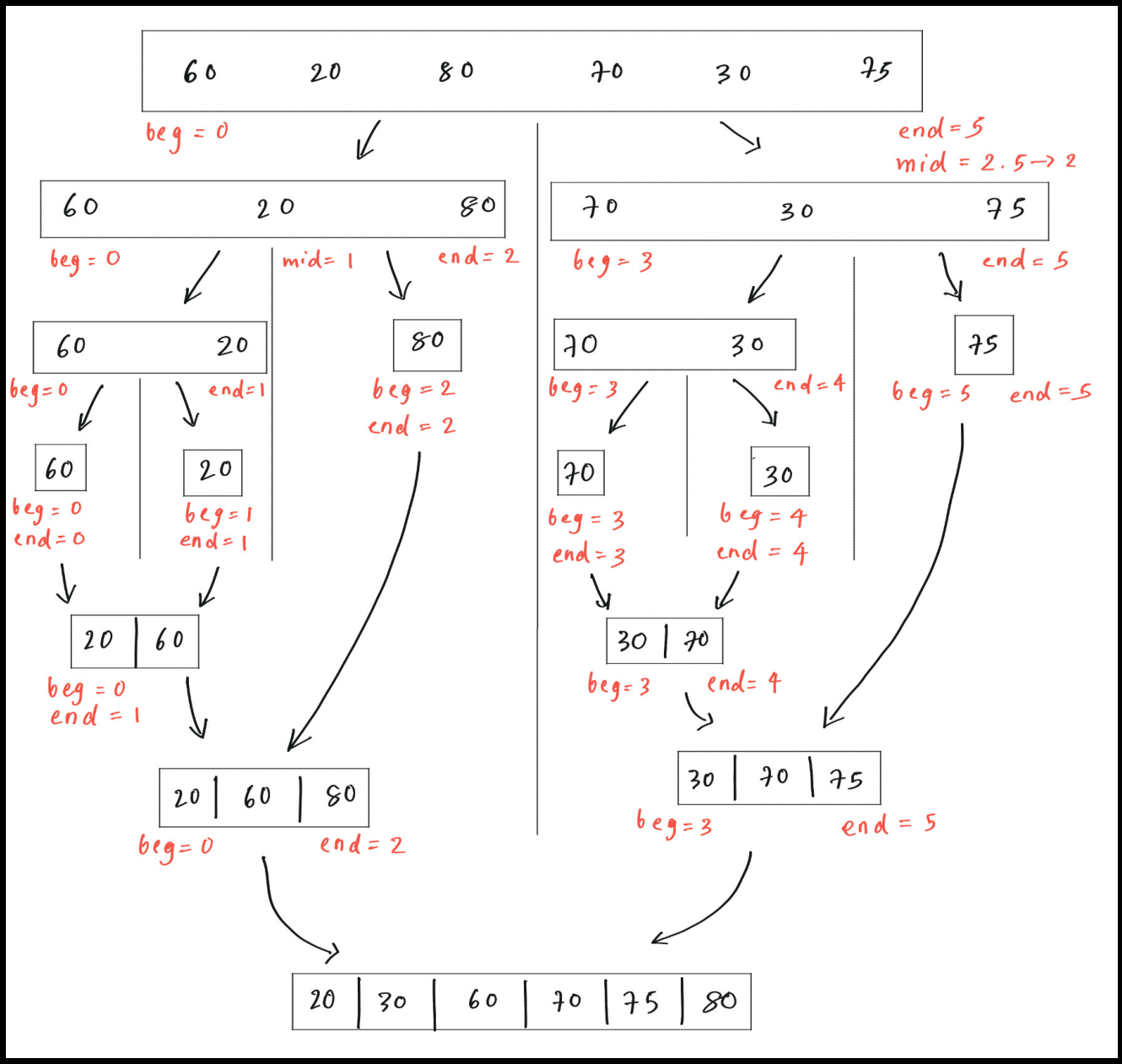
# 355. Iterative Merge Sort




CODE:

```
void MergeSort ( int A[], int n ){
    int p, i, l, mid, h ;
    for ( p = 2 ; p<=n ; p = p*2){
        for ( i=0 ; i+p-1<n ; i= i+p ){
            l = i ;
            h = i+p-1 ;
            mid = ⌊ (l+h) / 2 ⌋ ;
            merge ( A, l, mid, h ) ;
        }
    }
    if ( p/2 < n ){
        merge (A, 0, P/2-1, n-1) ; // If 9 elements or
    }                                 odd elements are
}                                     there.
```

# 357. Recursive Merge Sort




```
void MergeSort( int A[], int l, int h ){
    if( l < h ){
        mid = (l+h)/2;
        MergeSort( A, l, mid )
        MergeSort( A, mid+1, h);
        Merge ( A, l, mid, h);
    }
}
```

here,
beg = l
end = h
mid = mid

```
void Merge ( int A[], int L, int mid, int h ){
        int i, j, k ;
        int B[ h+1 ] ;
        i = k = L ;
        j = mid +1 ;


        while ( i <= mid  &&  j <= h ){
                if ( A[i] < A[j] )
                        B[k++] = A[i++] ;
                else
                        B[k++] = A[j++] ;
        }
        // For remmaining elements
        For ( ; i<= mid ; i++ )      // i initialised to wherever
                B[k++] = A[i] ;          it is right now.

        For ( ; j <= h ; j++ )       // j initialised to wherever
                B[k++] = A[j] ;          it is right now.

        for ( i= L ; i<=h ; i++ )
                A[i] = B[i] ;
}
```

* Time complexity (avg case) = $O(n \log_2 n)$
* As it uses recursion, STACK is required.

* Space required :

A[] = $n$

B[] = $n$

Stack = $\log_2 n$

---

$2n + \log_2 n$

* So, $(n + \log_2 n)$ is the extra space req. for merge sort.